



Report 3 - CUDA

September 30, 2026

0.1 2D Grayscale

```
@cuda.jit
def grayscale2D(src, dst):
    # where are we in the input?
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if y < src.shape[0] and x < src.shape[1]:
        g = np.uint8((src[y, x, 0] + src[y, x, 1] + src[y, x, 2]) / 3)
        dst[y, x, 0] = dst[y, x, 1] = dst[y, x, 2] = g

devSrc2 = cuda.to_device(img)
devDst2 = cuda.device_array_like(img)

#single blank kernel call with arbitrary gridSize 8
blankGridSize2 = ((width+8-1)//8, (height+8-1)//8)
grayscale2D[blankGridSize2, (8,8)](devSrc2, devDst2)

times = []
range_avg = 5
blockSides = range(1,33)
for blockSize in blockSides:
    avg = 0
    for i in range(range_avg):
        blockSize2 = (blockSize, blockSize)

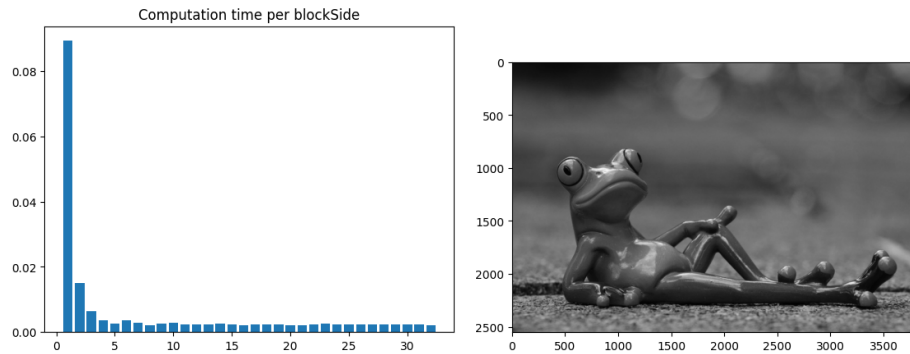
        #rounding up
        gridSizeX = (width+blockSize-1)//blockSize
        gridSizeY = (height+blockSize-1)//blockSize
        gridSize2 = (gridSizeX, gridSizeY)

        start=time.time()

        #kernel call
        grayscale2D[gridSize2, blockSize2](devSrc2, devDst2)

        cuda.synchronize()
        stop=time.time()
        avg+=stop-start
    avg/=range_avg
    times.append(avg)
```

- **2D best blockside: 8 with duration: 0.0010**
- vs.



- 1D best blockside: 128 with duration: duration 0.0046

0.2 Exercise1

- Hardware limit of 512 threads/block means $32 \times 32 = 1024$ doesn't fit as a block size.
- Maximizing SM: both 8×8 and 16×16 use all blocks of SM (
- However 8×8 blocks would mean 16 blocks/SM which doesn't fit the 8 blocks/SM limit. 16×16 is 4 which is fine.
- In the end only **16×16** fits. 32 threads/warp would mean there would be 8warps/block.

0.3 Exercise2

- $4 \times 128 = 512$
- $4 \times 256 = 1024$
- **$3 \times 512 = 1536$ this is the best config**
- $1 \times 1024 = 1024$

0.4 Exercise3

- $(2000 + 512 - 1) // 512 = 4$
- There will be 4 blocks of 512 threads, meaning **2048** threads in the grid.